

TP4 : Streams et expressions Lambda

Créer les classes suivantes :

1. La classe **Personne** avec :
 - 1.1. Attributs : nom (String), prenom (String) et age (int)
 - 1.2. Constructeur, getters, setters, et toString()
2. La classe **Etudiant** qui hérite de Personne, avec :
 - 2.1. Attribut supplémentaire : note (float)
 - 2.2. Constructeur, getters, setters, et toString()
3. La classe **Test1** avec la méthode main :
 - 3.1. Créer les 4 étudiants suivants :

```
Etudiant e1=new Etudiant("Karimi2", "Karim", 21, 15.5f);  
Etudiant e2=new Etudiant("Karimi1", "Karim", 20, 15);  
Etudiant e3=new Etudiant("Karimi3", "Karim", 21, 10);  
Etudiant e4=new Etudiant("Karimi4", "Karim", 19, 11);
```

- 3.2. Créer une ArrayList<Etudiant> et y ajouter les 4 étudiants.

3.3. Partie 1 — Découverte des Streams

- 3.3.1. Afficher tous les étudiants de la liste. (Méthodes : stream(), forEach())
- 3.3.2. Afficher uniquement les noms des étudiants. (Méthodes : stream(), map(), forEach())
- 3.3.3. Afficher les étudiants ayant une note ≥ 14 . (Méthodes : stream(), filter(), forEach())
- 3.3.4. Afficher les étudiants majeurs ($\text{âge} \geq 18$). (Méthodes : stream(), filter(), forEach())
- 3.3.5. Compter le nombre d'étudiants ayant une note > 10 . (Méthodes : stream(), filter(), count())

3.4. Partie 2 — Transformation et tri

- 3.4.1. Afficher la liste des noms en majuscules, triée par ordre alphabétique. (Méthodes : stream(), map(), sorted(), forEach())
- 3.4.2. Trier les étudiants par note croissante puis les afficher. (Méthodes : stream(), sorted(), forEach())
- 3.4.3. Trier les étudiants par âge décroissant. (Méthodes : stream(), sorted(), forEach())
- 3.4.4. Créer une nouvelle liste contenant uniquement les étudiants ayant réussi (note ≥ 10). (Méthodes : stream(), filter(), toList())
- 3.4.5. Récupérer une liste contenant les notes de tous les étudiants sans doublons. (Méthodes : stream(), map(), distinct(), toList())

3.5. Partie 3 — Opérations statistiques

- 3.5.1. Calculer la moyenne des notes des étudiants. (Méthodes : `stream()`, `mapToDouble()`, `average()`)
- 3.5.2. Trouver l'étudiant ayant la meilleure note. (Méthodes : `stream()`, `max()`)
- 3.5.3. Trouver l'étudiant le plus âgé sans utiliser `max()`. (Méthodes : `stream()`, `reduce()`)
- 3.5.4. Trouver l'étudiant ayant la plus petite note. (Méthodes : `stream()`, `min()`)
- 3.5.5. Calculer le total des âges des étudiants. (Méthodes : `stream()`, `mapToInt()`, `sum()`)
- 3.5.6. Vérifier si tous les étudiants ont une note ≥ 10 . (Méthodes : `stream()`, `allMatch()`)
- 3.5.7. Vérifier s'il existe au moins un étudiant avec une note < 8 . (Méthodes : `stream()`, `anyMatch()`)
- 3.5.8. Afficher le nom du premier étudiant dont la note est supérieure à 16. (Méthodes : `stream()`, `filter()`, `findFirst()`)

3.6. Partie 4 — Limitation, saut et collect

- 3.6.1. Afficher les 3 premiers étudiants de la liste. (Méthodes : `stream()`, `limit()`, `forEach()`)
- 3.6.2. Ignorer les 2 premiers étudiants et afficher les suivants. (Méthodes : `stream()`, `skip()`, `forEach()`)
- 3.6.3. Afficher les 2 meilleurs étudiants selon leur note. (Méthodes : `stream()`, `sorted()`, `limit()`, `forEach()`)
- 3.6.4. Créer une liste contenant uniquement les noms des étudiants. (Méthodes : `stream()`, `map()`, `collect(Collectors.toList())`)